

빅 데이터 분석을 위한 알고리즘

활용 빅 데이터 분석을 위한 알고리즘 소개 및 Python을 활용한 실습



CONTENTS

DATA Analysis

01

파이썬 프로그래밍 기초

파이썬에서 활용 가능한 자료의 형태와 기능들에 대해
학습함

02

데이터 수집

데이터 수집 과정을 소개하고 파이썬을 이용하여 데이터를
수집하는 방법을 학습함

03

데이터 전처리 및 시각화

분석 데이터의 구조 및 형태와 파이썬을 활용하여 분석 데이터를
생성하기 위한 방법들을 학습함

04

AI 알고리즘 맛보기

회귀모형과, Tree기반의 앙상블 모형에 대해 간단히
학습하고 파이썬으로 생성함

05

모형 평가 방법

다양한 AI 알고리즘들을 생성하고 평가하는 방법에 대해
학습함

- 컴퓨터가 스스로 판단하여 결과를 제시할 수 있게 하기 위해서는 기준에 따른 질문을 할 수 있어야 합니다.
- 이렇듯 컴퓨터에서 질문하는 대표적인 도구가 “비교 연산자”, 대답하는 형태가 “불리언(Boolean)” 입니다.

비교 연산자

■ 비교 연산자 요약

비교 연산자	내용
$x < y$	x가 y보다 작다
$x > y$	x가 y보다 크다
$x = y$	x가 y와 같다
$x \neq y$	x가 y와 같지 않다
$x \geq y$	x가 y보다 크거나 같다
$x \leq y$	x가 y보다 작거나 같다
x조건 or y조건	x조건과 y조건 둘 중에 하나만 참이면 참
x조건 and y조건	x조건과 y조건 모두 참이어야 참
not x조건	x조건이 거짓이면 참

불리언(Boolean)

■ 불리언 종류

- True / False

```
# 질문 : 10은 5보다 큰 가?
print(10 > 5)

# 질문 : "hello"와 "world"는 같은 가?
print("hello" == "world")

# 변수를 사용한 질문
my_age = 18
is_adult = (my_age >= 20)
print(f"나는 성인인가요? : {is_adult}")

# 결과 값의 데이터 타입 확인하기
print(type(is_adult))

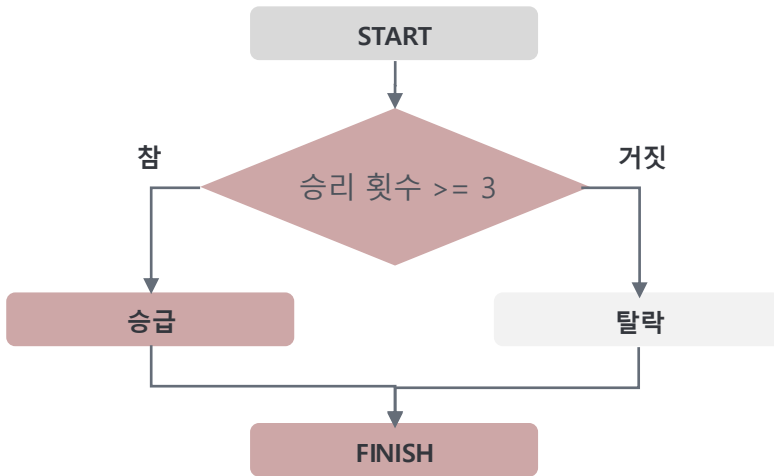
... True
False
나는 성인인가요? : False
<class 'bool'>
```

- 알고리즘을 개발하기 위해 '조건문'은 없어서는 안되는 부분입니다.
- 조건문은 사람이 어떤 상황에 놓여져 있을 때, 판단하는 것처럼 컴퓨터에게 특정 조건이 주어졌을 때 행동해야 하는 방법을 인식시켜주는 부분이라고 생각할 수 있습니다.

if문 (if / else)

■ 기본 개념

승급전에서 세판 이상 이기면 승급하고,
세 판 미만으로 이기면 떨어진다.



조건 연산자

■ 비교 연산자 (조건 연산자)

비교 연산자	내용
$x < y$	x가 y보다 작다
$x > y$	x가 y보다 크다
$x = y$	x가 y와 같다
$x \neq y$	x가 y와 같지 않다
$x \geq y$	x가 y보다 크거나 같다
$x \leq y$	x가 y보다 작거나 같다
x조건 or y조건	x조건과 y조건 둘 중에 하나만 참이면 참
x조건 and y조건	x조건과 y조건 모두 참이어야 참
not x조건	x조건이 거짓이면 참

- 알고리즘을 개발하기 위해 '조건문'은 없어서는 안되는 부분입니다.
- 조건문은 사람이 어떤 상황에 놓여져 있을 때, 판단하는 것처럼 컴퓨터에게 특정 조건이 주어졌을 때 행동해야 하는 방법을 인식시켜주는 부분이라고 생각할 수 있습니다.

if문 (if / else)

■ 기본 구조

if 조건문  콜론 입력!

수행할 문장 1

수행할 문장 2

동일한 들여쓰기로
한 블록을 생성

수행할 문장 3

→ 들여쓰기 위치 주의!

else : 콜론 입력!

수행할 문장 1

수행할 문장 2

동일한 들여쓰기로
한 블록을 생성

Python 코드

```
[14] # 전체 5판 중 게임에서 이긴 총 수
      win = 3

      if win >=3 :
          print("축하합니다. 승급하셨습니다.")
      else :
          print("안타깝게도 승급에 실패하셨습니다.")
```

축하합니다. 승급하셨습니다.

```
[15] # 전체 5판 중 게임에서 이긴 총 수
      win = 2

      if win >=3 :
          print("축하합니다. 승급하셨습니다.")
      else :
          print("안타깝게도 승급에 실패하셨습니다.")
```

안타깝게도 승급에 실패하셨습니다.

- 알고리즘을 개발하기 위해 '조건문'은 없어서는 안 되는 부분입니다.
- 조건문은 사람이 어떤 상황에 놓여져 있을 때, 판단하는 것처럼 컴퓨터에게 특정 조건이 주어졌을 때 행동해야 하는 방법을 인식시켜주는 부분이라고 생각할 수 있습니다.

elif 추가 문

■ 기본 구조

if 조건문A :

수행할 문장 1

수행할 문장 2

elif 조건문B :

수행할 문장 1

수행할 문장 2

이전 조건문인
조건문A가 거짓이고
조건문B이 참일 때, 수행

else :

수행할 문장 1

수행할 문장 2

Python 코드

변수 age가
0이상 9이하 이면 '영유아'/10이상 19이하 이면 '10대',
20이상 29이하 이면 '20대'/30이상 39이하 이면 '30대'
40이상 49이하 이면 '40대'/아무것도 해당하지 않으면 '기타'가

출력하는 조건문을 생성해라.

해답

- “조건문”에서 사용되는 elif는 불필요한 중첩 if-else문의 사용을 대체할 수 있습니다.

학점 배정하기 (중첩 if-else)

변수 score가
score가 90보다 크거나 같다면 "A",
score가 80보다 크거나 같다면 "B",
score가 70보다 크거나 같다면 "C",
그렇지 않다면 "재수강이 필요합니다."를

출력하는 조건문을 생성해라

```
▶ score = 85

if score >= 90:
    print("A 학점입니다!")
else:
    # 90점 미만인 경우, 다시 80점 이상인지 확인
    if score >= 80:
        print("B 학점입니다!")
    else:
        # 80점 미만인 경우, 다시 70점 이상인지 확인
        if score >= 70:
            print("C 학점입니다!")
        else:
            # 모든 조건에 해당하지 않는 경우
            print("재수강이 필요합니다.")
```

... B 학점입니다!

elif

```
score = 85

if score >= 90:
    print("A 학점입니다!")
elif score >= 80:
    # 85는 90보다 크지 않으니 첫 if는 통과.
    # 85는 80보다 크거나 같으니 여기서 True!
    # 아래 코드를 실행하고 조건문 전체를 빠져나갑니다.
    print("B 학점입니다!")
elif score >= 70:
    print("C 학점입니다!")
else:
    print("재수강이 필요합니다.")
```

B 학점입니다!

훨씬 간단하고 효율적으로 작성 가능 !!!

응용문제 (키오스크 프로그램)

```
# 1. 키오스크 프로그램

# 메뉴판 가격 변수에 저장
americano_price = 4000
latte_price     = 4500

# 상품 총 가격 계산
order_price = americano_price*2 + latte_price*1

# 최종 결제 금액 출력
print(f"최종 결제 금액은 {order_price}원 입니다.")

최종 결제 금액은 12500원 입니다.
```

```
# 키오스크 프로그램을 업그레이드 하기
# 만약 손님이 VIP라면 10% 할인을 해주고, 아니라면 정가를 받는다는 기능 추가

americano_price = 4000
latte_price     = 4500
is_vip = True

order_price = americano_price*2 + latte_price*1

if 
    

print(f"최종 결제 금액은 {order_price}원 입니다.")
```

업그레이드

응용문제 (로그인 프로그램 완성하기)

```
# 2. 로그인 프로그램 완성하기
# 아이디와 비밀번호가 모두 맞으면 '로그인 성공'
# 아이디는 일치하지만 비밀번호가 일치 하지 않으면 '비밀번호가 잘못 되었습니다.'
# 아이디가 일치하지 않으면 '존재하지 않는 아이디 입니다' 라는 프로그램을 작성
```

```
sv_id = "admin"
sv_pw = "1234"
```

```
in_id = "admin"
in_pw = "1234"
```

```
# 1단계 : 아이디가 일치하는지 확인
```

```
if
```

```
else :
```

- 파이썬에는 사용할 수 있는 다양한 자료형들이 존재합니다.
- 다양한 파이썬의 자료형들을 학습하면, 데이터 분석을 위한 코드작성 시 유용하게 활용할 수 있습니다.

항목	파이썬 사용 예	특징
리스트 (List)	리스트 명 = [요소1, 요소2, 요소3, ...]	여러 개의 숫자와 문자모음을 저장
튜플 (tuple)	튜플 명 = (요소1, 요소2, 요소3, ...)	리스트와 거의 같으나 값을 바꿀 수 없음
딕셔너리(dictionary)	딕셔너리명 = {Key1:Value1, Key2:Value2, ..}	대응 관계를 나타낼 수 있음
집합	집합명 = set(리스트, 문자 ... 등등),	자료의 순서가 없고, 중복을 허용 안함

리스트 (컨테이너의 개념)

- **리스트(컨테이너의 개념)**이 없다면, 회원이 1명이 아닌 100명인 경우? 100개의 변수를 만들어서 관리해야 합니다.
- **데이터가 많아질수록** 코드는 끝없이 길어지고, 탈퇴한 회원 아이디가 있는 경우, 해당 변수를 삭제해야 하기 때문에 변수명을 관리하는 것도 쉽지 않습니다.
- 컨테이너의 개념은 이름 그대로 **여러 개의 값들을 하나의 상자에 담아서 관리**할 수 있게 해주는 중요한 개념입니다.
- 그 중, 가장 많이 사용되는 기본 컨테이너의 개념인 **리스트**에 대해서 알아보겠습니다.

1.6 파이썬 자료의 형태(리스트)

01. 파이썬 프로그래밍 기초

- 숫자와 문자열을 모음으로 저장하는 경우 "리스트 자료형"을 사용할 수 있습니다.

리스트 생성하기

```
[1] a = []  
b = [1, 2, 3]  
c = ['Life', 'is', 'too', 'short']  
d = [1, 2, 'Life', 'is']  
e = [1, 2, ['Life', 'is']]  
print(a)  
print(b)  
print(c)  
print(d)  
print(e)
```

```
☞ []  
[1, 2, 3]  
['Life', 'is', 'too', 'short']  
[1, 2, 'Life', 'is']  
[1, 2, ['Life', 'is']]
```

리스트 인덱싱과 슬라이싱

```
[5] e = [1, 2, ['Life', 'is']]  
print(e[0])  
print(e[2])
```

```
☞ 1  
['Life', 'is']
```

```
[8] print(e[-1][0])  
print(e[-1][1])
```

```
☞ Life  
is
```

```
[9] a = [1, 2, 3, 4, 5]  
a[0:2]
```

```
☞ [1, 2]
```

```
[10] a = "12345"  
a[0:2]
```

```
☞ '12'
```

리스트 연산자

```
▶ a = [1, 2, 3]  
b = [4, 5, 6]  
print(a + b)  
print(a * 3)
```

```
☞ [1, 2, 3, 4, 5, 6]  
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

```
[13] a[2] = 4  
print(a)  
  
a[1:2] = ['a', 'b', 'c']  
print(a)
```

```
☞ [1, 2, 4]  
[1, 'a', 'b', 'c', 4]
```

```
[14] a[1:3] = []  
a
```

```
☞ [1, 'c', 4]
```

```
[16] a = [1, 2, 3]  
a.append(4)  
print(a)  
a = [1, 4, 3, 2]  
a.sort()  
print(a)  
a.reverse()  
print(a)
```

```
☞ [1, 2, 3, 4]  
[1, 2, 3, 4]  
[4, 3, 2, 1]
```

- 리스트와 문자열은 비슷한 특징을 가지고 있지만, 큰 차이점들이 존재합니다.

구분	문자열 (str)	리스트 (list)
공통점	순서(sequence)가 있는 데이터의 묶음	순서(sequence)가 있는 데이터의 묶음
인덱싱	str[0] -> 한 글자	list[0] -> 한 요소(element)
슬라이싱	str[1:3] -> 여러 글자	list[1:3] -> 여러 요소
차이점	한 번 만들면 내용을 바꿀 수 없음 (immutable)	내용을 마음대로 바꾸고 추가할 수 있음 (mutable)
내용물	오직 글자만 담을 수 있음	숫자, 문자열, 심지어 다른 리스트까지 무엇이든 담을 수 있음

in 연산자

```
▶ ids = ["Kuk", "Kim", "Lee", "Park", "Jang"]

# "Lee"가 ids 리스트 안에 포함되어 있는가 ?
print("Lee" in ids)

# "Han"이 ids 리스트 안에 포함되어 있는가 ?
print("Han" in ids)
```

```
ids = ["Kuk", "Kim", "Lee", "Park", "Jang"]
new = input("생성하고자 하는 아이디를 입력하세요: ")

# 만약 new id가 ids 리스트 안에 포함되어 있다면,
if new in ids :
    print("이미 사용 중인 아이디 입니다.")
else :
    print("사용 가능한 아이디 입니다.")
```

- 리스트와 문자열은 비슷한 특징을 가지고 있지만, 큰 차이점들이 존재합니다.

구분	문자열 (str)	리스트 (list)
공통점	순서(sequence)가 있는 데이터의 묶음	순서(sequence)가 있는 데이터의 묶음
인덱싱	str[0] -> 한 글자	list[0] -> 한 요소(element)
슬라이싱	str[1:3] -> 여러 글자	list[1:3] -> 여러 요소
차이점	한 번 만들면 내용을 바꿀 수 없음 (immutable)	내용을 마음대로 바꾸고 추가할 수 있음 (mutable)
내용물	오직 글자만 담을 수 있음	숫자, 문자열, 심지어 다른 리스트까지 무엇이든 담을 수 있음

in 연산자

```
▶ ids = ["Kuk", "Kim", "Lee", "Park", "Jang"]

# "Lee"가 ids 리스트 안에 포함되어 있는가 ?
print("Lee" in ids)

# "Han"이 ids 리스트 안에 포함되어 있는가 ?
print("Han" in ids)
```

```
ids = ["Kuk", "Kim", "Lee", "Park", "Jang"]
new = input("생성하고자 하는 아이디를 입력하세요: ")

# 만약 new id가 ids 리스트 안에 포함되어 있다면,
if new in ids :
    print("이미 사용 중인 아이디 입니다.")
else :
    print("사용 가능한 아이디 입니다.")
```

- 동일한 동작을 여러 번 반복해야 하는 경우, 코드를 한 줄씩 입력하는 것은 매우 비효율적인 일이며, 많은 '행' 데이터를 확인하거나 처리하기 위해서는 반복문을 반드시 활용할 수 있어야 합니다.
- 반복문에는 상황에 따라 while문 for문과 같은 문법을 사용합니다.

while문

■ 기본 구조

- while문은 조건문이 참인 '동안' 그 안에 속해 있는 문장들을 반복해서 수행하는 문법임

초기조건 (초기값)

while 종료(작동)조건 :

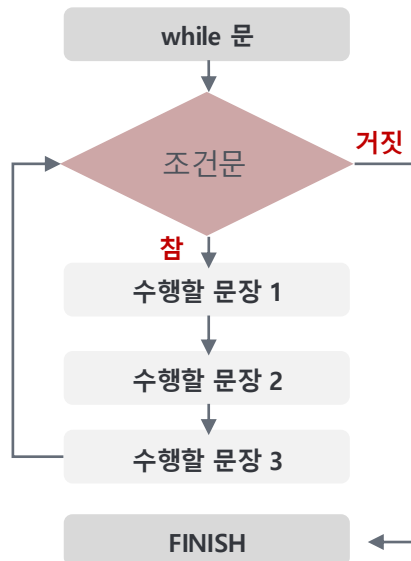
증 감 조 건

수행할 문장 1

수행할 문장 2

수행할 문장 3

증 감 조 건



Python 코드

```

# while 문
playGame = 0 # 초기 조건

while playGame <= 3: # 종료 조건
    print('게임을 %d 판 했습니다.' %playGame)

    if playGame == 3:
        print('마지막 게임이었습니다.')

    playGame = playGame + 1 # 증감 조건
    
```

게임을 0 판 했습니다.
 게임을 1 판 했습니다.
 게임을 2 판 했습니다.
 게임을 3 판 했습니다.
 마지막 게임이었습니다.

- 동일한 동작을 여러 번 반복해야 하는 경우, 코드를 한 줄씩 입력하는 것은 매우 비효율적인 일이며, 많은 '행' 데이터를 확인하거나 처리하기 위해서는 반복문을 반드시 활용할 수 있어야 합니다.
- 반복문에는 상황에 따라 while문 for문과 같은 문법을 사용합니다.

break / continue

- break
 - 반복문 내부에서 강제로 빠져나오고 싶을 때 사용하는 문법임
 - break를 만나면, 반복 횟수가 남았더라도 **즉시 반복문 전체를 완전히** 빠져나옴.
- continue
 - continue문에서 제시하는 특정 조건을 만족하는 경우, 그 하위 문장을 실행하지 않고 반복문의 시작지점으로 위치를 이동시킴
 - 반복문 전체를 빠져나가는 것은 아님.

while 조건문 :

if 조건문 :

break / continue

수행할 문장 1

Python 코드

```
# continue 사용
a = 0
while a < 6:
    a = a+1
    if a % 2 == 0:
        continue
    print(a)
```

```
# break 사용
item = 4
money = 30000
myitem = 0

while money > 1300 :
    print("아이템을 구매합니다.")
    item = item - 1
    myitem = myitem + 1
    money = money - 500
    print("나의 아이템은 %d개, 남은 돈은 %d원입니다." %(myitem, money))
    print("상점에 남은 아이템은 "+str(item)+"개 입니다.\n")
    if not item:
        print("아이템이 전부 소진되었습니다. 상점 문을 닫습니다.")
        break
```

- 동일한 동작을 여러 번 반복해야 하는 경우, 코드를 한 줄씩 입력하는 것은 매우 비효율적인 일이며, 많은 '행' 데이터를 확인하거나 처리하기 위해서는 반복문을 반드시 활용할 수 있어야 합니다.
- 반복문에는 상황에 따라 while문 for문과 같은 문법을 사용합니다.

for문

■ 기본 구조

- for문은 가장 기본적이고 대표적인 반복문으로 범위를 지정하여 범위에 해당하는 내용을 하나씩 순차적으로 대입하여 수행할 문장을 반복하는 문법임

for **변수** **in** **범위** **:**

수행할 문장 1

수행할 문장 2

수행할 문장 3

리스트/튜플/문자열
모두 사용 가능하며,
range함수를 활용하여
특정 숫자 범위를 지정
하기도 함

Python 코드

```
# 안녕하세요를 5번 출력하세요.  
for i in range(5):  
    print("안녕하세요")
```

```
# while 문 사용  
# i = 0  
# while i < 5:  
#     print("안녕하세요")  
#     i = i + 1
```

```
안녕하세요  
안녕하세요  
안녕하세요  
안녕하세요  
안녕하세요
```

```
# for문 기본구조  
test_list = ['one', 'two', 'three']  
for i in test_list:  
    print(i)
```

```
▶ # range 함수를 활용한 for문  
# range 함수는 range(시작 숫자, 마지막 숫자+1)을 작성하여 사용합니다.  
# 1부터 10까지 더하기  
sum = 0  
for i in range(1, 11):  
    sum = sum + i  
print(sum)
```


- 동일한 동작을 여러 번 반복해야 하는 경우, 코드를 한 줄씩 입력하는 것은 매우 비효율적인 일이며, 많은 '행' 데이터를 확인하거나 처리하기 위해서는 반복문을 반드시 활용할 수 있어야 합니다.
- 반복문에는 상황에 따라 while문 for문과 같은 문법을 사용합니다.

반복문과 조건문 활용 예시

■ 기본 구조

```
sample3 = pd.read_csv("sample3.csv")  
sample3.head()
```

	id	course_nm	status
0	1234	computer application	allowed
1	1235	computer application	allowed
2	1236	statistic	allowed
3	1237	statistic	allowed
4	1238	statistic	allowed

수강생이 30명 이상인 경우, "Large Room",
20명 이상인 경우, "Medium Room",
10명 이상인 경우 "Small Room"으로 배정하고,
수강인원이 10명 미만이어서 status가 "not allowed"인 경우,
"not assigned"으로 배정하는 변수를 생성해보자

Python 코드

해답